Class 07 — Decision Making: Tricky Practice Problems

Read from stdin · Write to stdout · Submit each solution to the code judge

For every problem, work through all four steps **in order**. The goal is not just a correct solution — it is to understand *why* the wrong solution fails and *why* the correct one is always right.

Steps for each problem

Step	What you do
1	Read the trap solution. Paste it into the judge, OR write your own first attempt.
2	Identify the error type. Explain what was wrong in your own words.
3	Write and submit the correct solution.
4	Write a short proof that your solution is always correct.

Error types

- **Syntax Error** — Python cannot even parse the code (wrong indentation, missing colon, typo in keyword, ...)
- **Logic Error** — Code runs without crashing but produces wrong answers for some inputs
- **Edge Case Missing** — Logic is mostly right but fails on a specific boundary input

---

Q1 · Domino Piling `[CF 50A · Easy]`

You have a rectangular board with **m** rows and **n** columns. A domino covers exactly **two adjacent cells** (side-by-side horizontally or vertically). What is the **maximum** number of dominoes you can fit on the board?

```
Input:  two integers m and n on one line
Output: one integer

  2  3  →  3
  3  3  →  4
  1  1  →  0
  4  4  →  8
```

Step 1 — Trap solution *(copy this into the judge, or write your own first attempt)*

```
m, n = map(int, input().split())
print(m * n)
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

What was wrong? *(Be specific — which input breaks it, and why?)*

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

Fill in the blanks to complete the proof.

```
Every domino covers exactly ___ cell(s).
A board with m×n cells has ___ cells total.
We can always pair up every two cells with a domino, except when m×n is ___.
In that case, one cell is left uncovered — it cannot form a pair.
So the maximum number of dominoes is _________________.
```

Q2 · Codeforces Checking `[CF 1791A · Easy]`

You are given a **single lowercase letter** c. Print `YES` if c appears somewhere in the word `codeforces`, and `NO` otherwise.

```
Input:  one character
Output: YES or NO

  c  →  YES
  a  →  NO
  s  →  YES
  z  →  NO
  e  →  YES      (appears twice in "codeforces" — but that's irrelevant)
```

Step 1 — Trap solution

```
c = input()
if c in "Codeforces":
    print("YES")
else:
    print("NO")
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

What was wrong? *(Which specific inputs fail? Why?)*

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

```
The letters that appear in "codeforces" are: ____________________________.
Python's `in` operator on a string checks whether the left operand is a
_________________ of the right operand. It is case-_________________.
The trap used "Codeforces", which contains uppercase ___ and ___.
So inputs ___ and ___ returned the wrong answer.
The fix: use "_____________" (all lowercase), which is exactly the word as given.
```

Q3 · Yogurt Sale `[CF 1955A · Easy]`

A shop sells yogurt at **a** burles each. There is also a promotion: buy **two** yogurts together for **b** burles. Maxim wants exactly **n** yogurts. Find the **minimum** he must spend.

```
Input:  three integers n  a  b  on one line
Output: one integer (minimum cost)

  2  5  9  →  9    (pair is cheaper than 5+5=10)
  3  5  9  →  14   (one pair at 9, one single at 5)
  1  5  9  →  5    (only one yogurt, no pair possible)
  4  5  7  →  14   (two pairs at 7 each = 14, cheaper than 4×5=20)
  3  5  11 →  15   (pair costs 11 > 2×5=10, so buy all individually)
```

Step 1 — Trap solution

```
n, a, b = map(int, input().split())
print((n // 2) * b + (n % 2) * a)
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

What was wrong? *(Which example above exposes the bug?)*

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

```
The promotion is worth using only when b < ___ × a.
When we use the promotion: every pair of yogurts costs ___ burles.
When we don't: every yogurt costs ___ burles, so a pair costs ___.
For n yogurts, we form ___ complete pairs, plus possibly ___ leftover.
The leftover (if any) must be bought individually at ___ burles.
```

Q4 · Minimal Square `[CF 1360A · Easy]`

You have two rectangles: one of size **a × 1** and one of size **b × 1** (both are 1 unit wide). Find the side length of the **smallest square** into which both rectangles fit without overlapping. You may rotate the rectangles.

```
Input:  two integers a and b on one line
Output: one integer (the side of the minimum square)

  2  3  →  3
  3  1  →  3
  5  5  →  5
  1  1  →  2
  4  1  →  4
```

Step 1 — Trap solution

```
a, b = map(int, input().split())
print(max(a, b))
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

Which test case above exposes the bug? Why does `max(a, b)` fail there?

```
_____________________________________________________________________
_____________________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

```
Each rectangle is 1 unit tall. To fit both without overlap, stack them one above
the other. This arrangement needs:
  - Width  ≥  the longer rectangle  →  width ≥ ___
  - Height ≥  ___ (one rectangle per row)

So the minimum square side is max(___, ___).

Why is stacking always at least as good as any other arrangement?
(Hint: any other arrangement either uses more width or more height.)


_____________________________________________________________________
```

Q5 · Calculating Function `[CF 486A · Easy]`

Define f(n) = 1 − 2 + 3 − 4 + 5 − 6 + ... + (−1)^(n+1) × n.

In other words: the terms alternate in sign, starting positive.

```
Input:  one integer n  (1 ≤ n ≤ 10^8)
Output: one integer f(n)

  1  →   1
  2  →  -1
  3  →   2
  4  →  -2
  5  →   3
  6  →  -3
```

Step 1 — Trap solution

```
n = int(input())
print(n * (n + 1) // 2)
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

What formula does the trap compute? Which value of n first shows it is wrong?

```
_____________________________________________________________________
_____________________________________________________________________
```

Step 3 — Correct solution

(Hint: group the terms into pairs (1–2), (3–4), ... and handle the leftover term when n is odd.)

```
# your solution here
```

Step 4 — Proof of correctness

```
Group consecutive pairs: (1−2) + (3−4) + … Each pair equals ___.

Case A — n is even:
  There are ___ complete pairs.
  f(n) = ___ × ___ = ___.

Case B — n is odd:
  There are ___ complete pairs, plus the final unpaired term +n.
  f(n) = ___ × ___ + n = ___.

Verify with n=5: (1−2)+(3−4)+5 = ___ + ___ + 5 = ___.
Formula gives: _____________________________.
```

Q6 · Theatre Square `[CF 1A · Medium]`

A city's main square is **n × m** metres. The mayor wants to pave it entirely with square granite flagstones of size **a × a** metres. Flagstones may extend beyond the square's edge. What is the **minimum number of flagstones** needed?

```
Input:  three integers n  m  a  on one line
Output: one integer

  6  6  4  →  4      (2×2 grid of 4×4 stones covers 8×8 > 6×6)
  2  3  1  →  6
  3  3  3  →  1
  5  4  3  →  4      (ceil(5/3)=2 rows, ceil(4/3)=2 columns → 4 stones)
```

Step 1 — Trap solution

```
n, m, a = map(int, input().split())
print((n * m) // (a * a))
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

Trace the trap on input `6 6 4`. What does it print? What is the correct answer? Why is floor division wrong here?

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

(Hint: how do you compute ceil(n/a) using only integer arithmetic — no `math` module?)

```
# your solution here
```

Step 4 — Proof of correctness

```
We need ceil(n/a) flagstones to cover n metres in one direction.
Using integer arithmetic: ceil(n/a) = (n + a − 1) // a.

Why does this formula give the ceiling?
  n // a gives the floor. We want to round up whenever n is not a
  multiple of a. Adding (a − 1) before dividing ensures that any
  remainder ≥ 1 causes the division to round up. Show this:

  If n = k·a (exact fit):        (k·a + a − 1) // a  =  ___
  If n = k·a + r (1 ≤ r < a):   (k·a + r + a − 1) // a  =  ___

Total flagstones = ceil(n/a) × ceil(m/a)  because the rows and columns
are independent. Each row needs ___ flagstones; there are ___ such rows.
```

Q7 · Restoring Three Numbers `[CF 1154A · Easy]`

Vasya had three positive integers a_1, a_2, a_3. He added them in pairs and wrote down the three sums:

$$b_1 = a_1 + a_2$$
$$b_2 = a_1 + a_3$$
$$b_3 = a_2 + a_3$$

Given b_1, b_2, b_3, recover a_1, a_2, and a_3. It is guaranteed that a valid solution exists.

```
Input:  three integers b₁ b₂ b₃ on one line
Output: three integers a₁ a₂ a₃ on one line

   3   5   6  →  1 2 4      (check: 1+2=3 ✓  1+4=5 ✓  2+4=6 ✓)
  14  18  12  →  10 4 8     (check: 10+4=14 ✓  10+8=18 ✓  4+8=12 ✓)
   2   2   2  →  1 1 1
```

Step 1 — Trap solution

```
b1, b2, b3 = map(int, input().split())
print(b1 + b2 - b3, b1 + b3 - b2, b2 + b3 - b1)
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

Trace the trap on input `3 5 6`. What does it print? What should it print?

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

```
Start from: b₁ + b₂ + b₃  =  (a₁+a₂) + (a₁+a₃) + (a₂+a₃)
                          =  ___ × (a₁ + a₂ + a₃)

So a₁ + a₂ + a₃  =  _____________________________.

Now:  a₁  =  (a₁+a₂+a₃) − b₃  =  _____________________________
      a₂  =  (a₁+a₂+a₃) − b₂  =  _____________________________
      a₃  =  (a₁+a₂+a₃) − b₁  =  _____________________________

Equivalently: a₁ = (b₁+b₂−b₃) // ___ .
The trap forgot to divide by ___, so it printed results ___ times too large.
```

Q8 · Divisor Game `[LeetCode 1025 · Easy — rewritten for stdin/stdout]`

Alice and Bob play a game starting with the integer **n**. Alice goes first. On each turn, the current player picks a number **x** such that:

- 0 < x < n
- x divides n evenly (n % x == 0)

The player **replaces n with n − x** and passes the turn. A player who **cannot move** (because no valid x exists) **loses**.

Both play optimally. Who wins?

```
Input:  one integer n  (1 ≤ n ≤ 1000)
Output: Alice  or  Bob

  1  →  Bob      (Alice has no valid x; n=1 has no divisor < 1)
  2  →  Alice    (Alice picks x=1, n becomes 1, Bob cannot move)
  3  →  Bob      (Alice must pick x=1, n becomes 2, Alice wins... wait — trace it)
  4  →  Alice
  6  →  Alice
```

(Tip: trace n=3 carefully by hand before coding.)

Step 1 — Trap solution

```
n = int(input())
if n % 2 == 1:
    print("Alice")
else:
    print("Bob")
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

Which inputs above immediately disprove the trap? What is the correct pattern?

```
_____________________________________________________________
_____________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

Fill in the reasoning:

```
n=1:  No valid x exists for Alice → Bob wins.
n=2:  Alice picks x=___ → n becomes ___ → Bob has no move → Alice wins.
n=3:  All divisors of 3 less than 3 are: ___. Alice must pick x=___.
      n becomes ___. Now it is Bob's turn with n=___  → Bob wins.
      So n=3 → ___ wins.

Pattern for even n:
  Every even n has x=1 as a divisor (and 1 < n, so it is valid).
  Alice picks x=1 → n becomes n-1 which is ___.
  Now Bob faces an odd number. Every divisor of an odd number is ___.
  So n-x is odd - odd = ___ → Bob must give Alice an ___ number.
  Alice can always pick x=1 again → Bob always receives odd → Bob loses.
  Therefore: even n → ___ wins.

Pattern for odd n:
  All divisors of an odd number are ___ (odd divisors only).
  So n-x is odd - odd = ___ → Alice hands Bob an ___ number → Bob wins.
  Therefore: odd n → ___ wins.
```

Q9 · Nim Game `[LeetCode 292 · Easy — rewritten for stdin/stdout]`

There is a pile of **n** stones. You and your opponent take turns; you go first. On each turn, the current player removes **1, 2, or 3** stones from the pile. The player who removes the **last stone wins**.

Both play optimally. Can you win?

```
Input:  one integer n  (1 ≤ n ≤ 10^9)
Output: Win  or  Lose

  1  →  Win     (take 1 stone immediately)
  2  →  Win     (take 2)
  3  →  Win     (take 3)
  4  →  Lose    (whatever you take, opponent finishes the pile)
  5  →  Win     (take 1, leave 4 for opponent)
  8  →  Lose
  9  →  Win
```

Step 1 — Trap solution

```
n = int(input())
if n % 3 != 0:
    print("Win")
else:
    print("Lose")
```

Step 2 — Diagnose the error

Error type (circle one): **Syntax Error** | **Logic Error** | **Edge Case Missing**

Use the examples above to find an input where the trap gives the wrong answer.

```
Input that breaks it: n = ___
Trap prints: ___    Correct answer: ___
The trap uses modulo ___ — what modulo should be used, and why?
_____________________________________________________________________
```

Step 3 — Correct solution

```
# your solution here
```

Step 4 — Proof of correctness

```
Define: a position is LOSING if the player whose turn it is will lose
        with optimal play; otherwise WINNING.

Base cases (verify each):
  n=1 → ___ (take 1, pile empty, opponent has nothing → you win)
  n=2 → ___
  n=3 → ___
  n=4 → whatever you take (1, 2, or 3), you leave ___, ___, or ___ stones.
         All of those are ___ positions → n=4 is a ___ position.

Inductive step:
  Claim: n is LOSING if and only if n % 4 == 0.

  Case A — n % 4 == 0:
    You can take 1, 2, or 3 stones. The remaining pile is n-1, n-2, or n-3.
    Each of these has remainder ___ (≠ 0) modulo 4 → they are all ___ positions
    → your opponent wins from any of them → n is ___.

  Case B — n % 4 == r  (r = 1, 2, or 3):
    Take exactly ___ stones. Remaining pile = n - r, which is divisible by 4
    → that is a ___ position for your opponent → you win → n is ___.

Therefore: n % 4 == 0 → Lose; otherwise → Win.
```

End of Class 07 Worksheet